

ECE 60146 HW 06

Andrea Goh

February 2025

1 Playing with Skip Connections

1.1 Task 1: Comparing the use of a 1×1 Kernel

On page 16 of week 6 lecture, the definition for SkipBlock contain convolutional operators that uses 1×1 kernel. Convolutions that are carried out with a 1 pixel kernel, is essentially a scaling operator. Each pixel in the input image is multiplied by the kernel value to generate the output pixel. The purpose SkipBlock uses such a convolution is for two reasons, to change the number of channels, and to downsample the image.

In line (I) of the SkipBlock definition code, the convolution looks like: `self.in2out = nn.Conv2d(in_ch, out_ch, 1)`. This convolution has a 1 pixel kernel and a stride of 1, which is default. Since the stride is 1, each pixel in each of the `in_ch` channels of the input image is multiplied by the kernel pixel value to obtain each of the `out_ch` channels. Thus, the purpose of this line is to change the number of channels from `in_ch` to `out_ch`.

In line (K) and (L), the convolution looks like: `self.downsampler1 = nn.Conv2d(in_ch, in_ch, 1, stride=2)` and `self.downsampler1 = nn.Conv2d(out_ch, out_ch, 1, stride=2)`. This convolution has a 1 pixel kernel and a stride of 2. In this case, the number of input channels and output channels remain the same. However, the stride of 2 means that every other pixel in the input image is being multiplied by the kernel value to obtain the output image. Therefore, the purpose of this 1 pixel convolution is to downsize the size of the image by half.

2 Comparing two Methods for Downsizing

In this section, we will be comparing two different methods for downsizing. Net1 uses a stride length of 2 with a kernel size of 1 to perform downsizing. This means it does not aggregate the information from neighboring pixels. Rather, it downsizes by skipping every other pixel. On the other hand, Net2 uses `MaxPool2d(2,2)`, which essentially selects the maximum value within each 2×2 patch. Thus, it preserves the most dominant features.

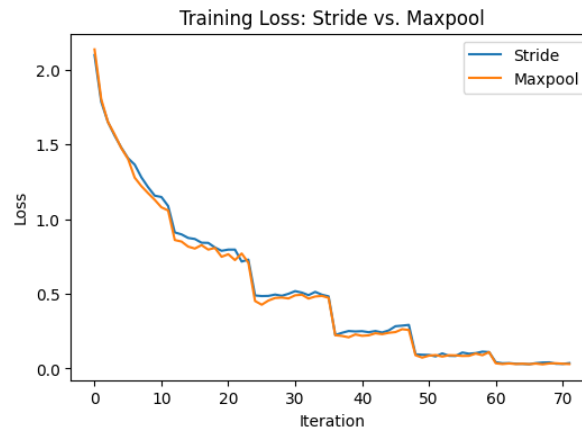


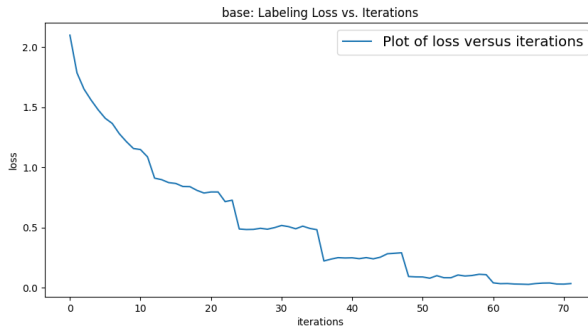
Figure 1: Running Loss Comparison

	Stride	Maxpool
plane	82%	86%
car	88%	91%
bird	73%	70%
cat	62%	58%
deer	77%	79%
dog	69%	75%
frog	86%	83%
horse	84%	83%
ship	89%	88%
truck	88%	86%

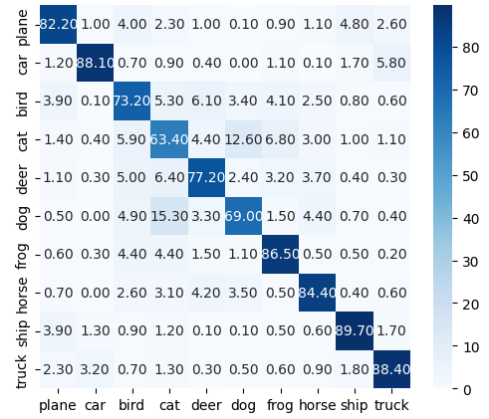
Table 1: Comparison of class accuracies for the different downsizing methods

	Stride	Maxpool
Overall Accuracy	80%	80%

Table 2: Overall Accuracy of both downsizing methods

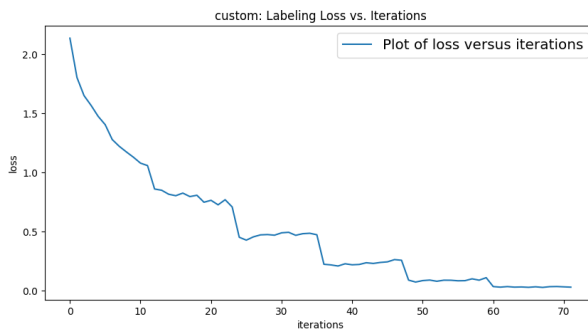


(a) Net1: Loss vs. Iterations

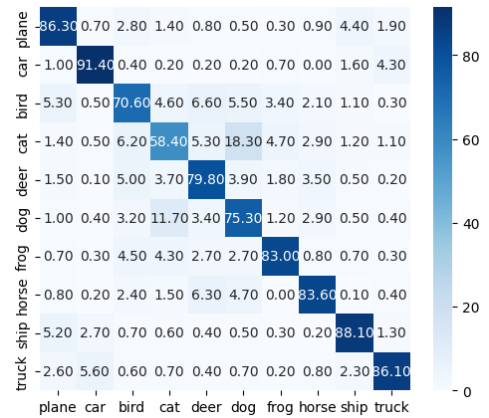


(b) Net1: Confusion Matrix

Figure 2: fig: Results from Net1



(a) Net2: Loss vs. Iterations



(b) Net2: Confusion Matrix

Figure 3: fig: Results from Net2

Both downsizing methods result in the same overall performance.

3 Task 2: Skip Connections with MS-COCO

In this section, a network, HW5Net, with at least 54 learnable layers is created to classify images from the custom MSCOCO dataset. The following classes are used: [airplane, bus, cat, dog, pizza]. Samples from the dataset are shown in figures 4 and 5.

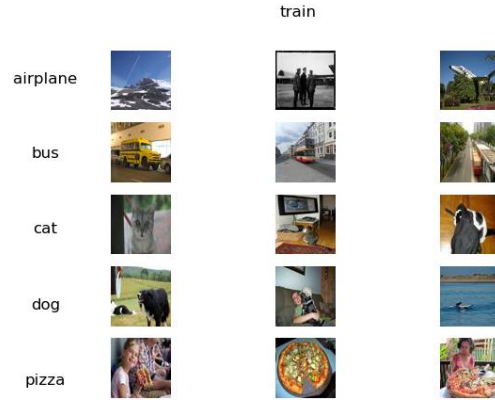


Figure 4: Training Dataset

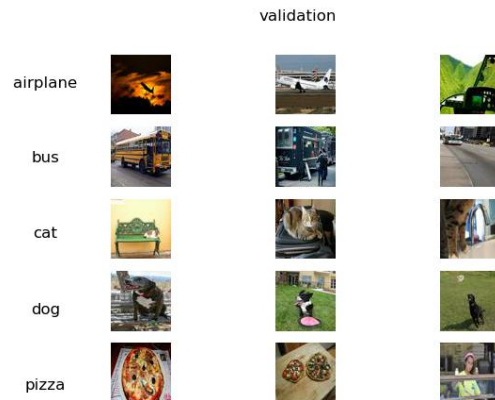


Figure 5: Validation Dataset

BMEnet from DLStudio was used as a starter for the network. In this network, instead of using Stochastic Gradient Descent like BMEnet does, Adam optimizer is used instead. The total number of layers used in this network is 54.

Class	Accuracy
airplane	79%
bus	64%
cat	45%
dog	42%
pizza	52%

Table 3: Class accuracies of HW5Net

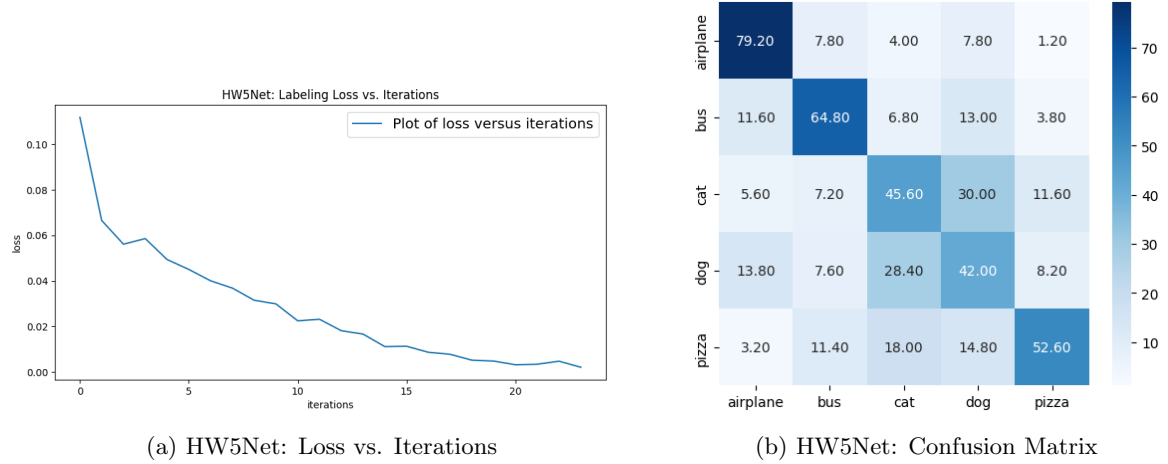


Figure 6: fig: Results from HW5Net

HW5Net achieved an overall accuracy of 56%. The confusion matrix reveals that the airplane class is the easiest to distinguish. In contrast, distinguishing between the cat and dog classes is more challenging. This is expected, as cats and dogs are the most similar among the five classes, both being four-legged animals.

4 Code: Task 1

```

1 from DLStudio import *
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4 from torch.utils.data import DataLoader, Dataset
5
6 dls = DLStudio(
7     dataroot = "./data/CIFAR-10/",
8     image_size = [32,32],
9     path_saved_model = "./saved_model",
10    momentum = 0.9,
11    learning_rate = 1e-4,
12    epochs = 6,
13    batch_size = 4,
14    classes = ('plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', '
15              ship', 'truck'),
16    use_gpu = True,
17)
18
19 class CustomDataloader(Dataset):
20     def __init__(self, root, train=True):
21         self.categories = [cat for cat in os.listdir(root)]
22         self.img_paths = []
23         self.labels = []
24         if train:
25             self.img_paths = [os.path.join(root, 'train', img) for img in os.listdir(
26                 root, 'train')]
27             self.labels = np.load(f'{root}/train_labels.npy', allow_pickle=True)
28         else:

```

```

28         self.img_paths = [os.path.join(root, 'validation', img) for img in os.
29                             listdir(root, 'validation')]
30         self.labels = np.load(f'{root}/val_labels.npy', allow_pickle=True)
31
32     def __len__(self):
33         return len(self.img_paths)
34
35     def __getitem__(self, index):
36         img_path = self.img_paths[index]
37         self.img = Image.open(img_path)
38         transform = transforms.Compose([
39             transforms.Grayscale(num_output_channels=3), # some images are Grayscale but
40                 we want images to be 3 X 64 X 64
41             transforms.ToTensor(),
42             transforms.Normalize((0.5,0.5,0.5),(0.5,0.5,0.5))
43         ])
44         transformed_img = transform(self.img)
45         label = self.labels[index]
46         return (transformed_img, label)
47
48 class CustomSkipConnections(DLStudio.BMEnet):
49     #####
50     ##### Start: Copy from DLStudio with minor changes #####
51
52     def run_code_for_training(self, net, net_name):
53         filename_for_out = "performance_numbers_" + net_name + ".txt"
54         FILE = open(filename_for_out, 'w')
55         net = copy.deepcopy(net)
56         net = net.to(self.dl_studio.device)
57         criterion = nn.CrossEntropyLoss()
58         optimizer = optim.SGD(net.parameters(), lr=self.dl_studio.learning_rate,
59                                momentum=self.dl_studio.momentum)
60         print("\n\nStarting training loop...")
61         start_time = time.perf_counter()
62         loss_tally = []
63         elapsed_time = 0.0
64         for epoch in range(self.dl_studio.epochs):
65             print("")
66             running_loss = 0.0
67             for i, data in enumerate(self.dl_studio.train_data_loader):
68                 inputs, labels = data
69                 if i % 1000 == 999:
70                     current_time = time.perf_counter()
71                     elapsed_time = current_time - start_time
72                     print("\n\n[epoch:%d/%d iter=%4d elapsed_time=%5d secs] Ground
73                         Truth: " %
74                             (epoch+1, self.dl_studio.epochs, i+1, elapsed_time) +
75                             ' '.join('%10s' % self.dl_studio.class_labels[labels[j]] for j
76                                     in range(self.dl_studio.batch_size)))
77                     inputs = inputs.to(self.dl_studio.device)
78                     labels = labels.to(self.dl_studio.device)
79                     optimizer.zero_grad()
80                     outputs = net(inputs)
81                     loss = criterion(outputs, labels)
82                     running_loss += loss.item()
83                     if i % 1000 == 999:
84                         _, predicted = torch.max(outputs.data, 1)
85                         print("[epoch:%d/%d iter=%4d elapsed_time=%5d secs] Predicted
86                             Labels: " %
87                             (epoch+1, self.dl_studio.epochs, i+1, elapsed_time) +
88                             ' '.join('%10s' % self.dl_studio.class_labels[predicted[j]] for j
89                                     in range(self.dl_studio.batch_size)))
89                     avg_loss = running_loss / float(1000)
90                     loss_tally.append(avg_loss)
91                     print("[epoch:%d/%d iter=%4d elapsed_time=%5d secs] Loss: %.3f"
92                           %
93                               (epoch+1, self.
94                                   dl_studio.epochs,
95                                   i+1,
96                                   elapsed_time,
97                                   avg_loss))
98
99             FILE.write("%.3f\n" % avg_loss)
100             FILE.flush()

```

```

89         running_loss = 0.0
90         loss.backward()
91         optimizer.step()
92     print("\nFinished Training\n")
93     self.save_model(net, net_name)
94     plt.figure(figsize=(10,5))
95     plt.title(net_name + ": Labeling Loss vs. Iterations")
96     plt.plot(loss_tally)
97     plt.xlabel("iterations")
98     plt.ylabel("loss")
99     plt.legend(["Plot of loss versus iterations"], fontsize="x-large")
100    plt.savefig(f"loss_vs_iterations_{net_name}.png")
101
102    def save_model(self, model, net_name):
103        torch.save(model.state_dict(), self.dl_studio.path_saved_model + '_' + net_name)
104
105    def run_code_for_testing(self, net, net_name):
106        net.load_state_dict(torch.load(self.dl_studio.path_saved_model + '_' + net_name))
107        net = net.eval()
108        net = net.to(self.dl_studio.device)
109        filename_for_results = "classification_results_" + net_name + str(self.dl_studio
        .epochs) + ".txt"
110        FILE = open(filename_for_results, 'w')
111        correct = 0
112        total = 0
113        confusion_matrix = torch.zeros(len(self.dl_studio.class_labels), len(self.
        dl_studio.class_labels))
114        class_correct = [0] * len(self.dl_studio.class_labels)
115        class_total = [0] * len(self.dl_studio.class_labels)
116        with torch.no_grad():
117            for i, data in enumerate(self.dl_studio.test_data_loader):
118                ## data is set to the images and the labels for one batch at a time:
119                images, labels = data
120                images = images.to(self.dl_studio.device)
121                labels = labels.to(self.dl_studio.device)
122                if i % 1000 == 999:
123                    print("\n\n[i=%d]: Ground Truth: " % (i+1) + ', '.join('%5s' %
                        self.dl_studio.class_labels[labels[j]] for j in range(self.
                        dl_studio.batch_size)))
124                outputs = net(images)
125                _, predicted = torch.max(outputs.data, 1)
126                if i % 1000 == 999:
127                    print("[i=%d]: Predicted Labels: " % (i+1) + ', '.join('%5s' % self.
                        dl_studio.class_labels[predicted[j]] for j in range(self.
                        dl_studio.batch_size)))
128                logger = logging.getLogger()
129                old_level = logger.level
130
131                for label, prediction in zip(labels, predicted):
132                    confusion_matrix[label][prediction] += 1
133                total += labels.size(0)
134                correct += (predicted == labels).sum().item()
135                ## comp is a list of size batch_size of "True" and "False" vals
136                comp = predicted == labels
137                for j in range(self.dl_studio.batch_size):
138                    label = labels[j]
139                    class_correct[label] += comp[j].item()
140                    class_total[label] += 1
141            for j in range(len(self.dl_studio.class_labels)):
142                print('Prediction accuracy for %5s: %2d%%' % (self.dl_studio.class_labels[
                    j], 100 * class_correct[j] / class_total[j]))
143                FILE.write('\n\nPrediction accuracy for %5s: %2d%%\n' % (self.dl_studio.
                    class_labels[j], 100 * class_correct[j] / class_total[j]))
144            print("\n\nOverall accuracy of the network on the 10000 test images: %d%%" %
                (100 * correct / float(total)))
145            FILE.write("\n\nOverall accuracy of the network on the 10000 test images: %d
                %%\n" % (100 * correct / float(total)))
146            print("\n\nDisplaying the confusion matrix:\n")
147            FILE.write("\n\nDisplaying the confusion matrix:\n\n")
148            out_str = " "
149            for j in range(len(self.dl_studio.class_labels)): out_str += "%7s" % self.
                dl_studio.class_labels[j]

```

```

150     print(out_str + "\n")
151     FILE.write(out_str + "\n\n")
152     for i,label in enumerate(self.dl_studio.class_labels):
153         out_percents = [100 * confusion_matrix[i,j] / float(class_total[i])
154                         for j in range(len(self.dl_studio.
155                                         class_labels))]
156
157         out_percents = ["%.2f" % item.item() for item in out_percents]
158         out_str = "%6s:  " % self.dl_studio.class_labels[i]
159         for j in range(len(self.dl_studio.class_labels)): out_str += "%7s" %
160             out_percents[j]
161         print(out_str)
162         FILE.write(out_str + "\n")
163     FILE.close()
164     self.class_correct = class_correct
165     self.class_total = class_total
166     self.correct = correct
167     self.total = total
168     self.display_conf_matrix(confusion_matrix, net_name)
169
170 ##### End: Copy from DLStudio with minor changes #####
171 #####
172
173     def display_conf_matrix(self, confusion_matrix, net_name):
174         accuracy_matrix = []
175         for i,label in enumerate(self.dl_studio.class_labels):
176             out_percents = [100 * confusion_matrix[i,j] / float(self.class_total[i])
177                             for j in range(len(self.dl_studio.
178                                             class_labels))]
179
180             out_percents = [item.item() for item in out_percents]
181             accuracy_matrix.append([out_percents[j] for j in range(len(self.dl_studio.
182                             class_labels))])
183
184         plt.figure(figsize=(6,5))
185         sns.heatmap(np.array(accuracy_matrix, dtype=float), annot=True, fmt='%.2f', cmap=
186             'Blues', cbar=True,
187                     xticklabels=self.dl_studio.class_labels, yticklabels=self.dl_studio.
188                         class_labels)
189         plt.savefig(f'{net_name}_Confusion_matrix.png', bbox_inches='tight')
190
191 #####
192 ##### Start: Copy from DLStudio with minor changes #####
193 #####
194     class CustomSkipBlock(nn.Module):
195         def __init__(self, in_ch, out_ch, downsample=False, skip_connections=True):
196             super(CustomSkipConnections.CustomSkipBlock, self).__init__()
197             self.downsample = downsample
198             self.skip_connections = skip_connections
199             self.in_ch = in_ch
200             self.out_ch = out_ch
201             self.conv1 = nn.Conv2d(in_ch, in_ch, 3, stride=1, padding=1)
202             self.conv2 = nn.Conv2d(in_ch, out_ch, 3, stride=1, padding=1)
203             self.bn1 = nn.BatchNorm2d(in_ch)
204             self.bn2 = nn.BatchNorm2d(out_ch)
205             self.in2out = nn.Conv2d(in_ch, out_ch, 1)
206             if downsample:
207                 ## Setting stride to 2 and kernel_size to 1 amounts to retaining every
208                 ## other pixel in the image -- which halves the size of the image:
209                 self.downsampler1 = nn.MaxPool2d(2,2)
210                 self.downsampler2 = nn.MaxPool2d(2,2)
211
212         def forward(self, x):
213             identity = x
214             out = self.conv1(x)
215             out = self.bn1(out)
216             out = nn.functional.relu(out)
217             out = self.conv2(out)
218             out = self.bn2(out)
219             out = nn.functional.relu(out)
220             if self.downsample:
221                 identity = self.downsampler1(identity)
222                 out = self.downsampler2(out)
223             if self.skip_connections:
224                 if (self.in_ch == self.out_ch) and (self.downsample is False):
225                     out = out + identity
226                 elif (self.in_ch != self.out_ch) and (self.downsample is False):
227                     identity = self.in2out( identity )

```

```

217         out = out + identity
218         elif (self.in_ch != self.out_ch) and (self.downsample is True):
219             out = out + torch.cat((identity, identity), dim=1)
220         return out
221 ##### End: Copy from DLStudio with minor changes #####
222 #####
223
224
225 def compare_loss():
226     with open('performance_numbers_base.txt', 'r') as file:
227         running_loss1 = [float(line.strip()) for line in file]
228     with open('performance_numbers_custom.txt', 'r') as file:
229         running_loss2 = [float(line.strip()) for line in file]
230     plt.figure(figsize=(6,4))
231     plt.plot(running_loss1, label='Stride')
232     plt.plot(running_loss2, label='Maxpool')
233     plt.title('Training Loss: Stride vs. Maxpool')
234     plt.xlabel('Iteration')
235     plt.ylabel('Loss')
236     plt.legend()
237     plt.savefig('Training Loss Comparison.png', bbox_inches='tight')
238
239 # Task 1
240 bme_net = dls.BMEnet(dls, skip_connections=True, depth=8)
241 bme_net_custom = CustomSkipConnections(dls)
242
243 bme_net.load_cifar_10_dataset()
244 bme_net_custom.load_cifar_10_dataset()
245
246 # display network properties
247 number_of_learnable_params = sum(p.numel() for p in bme_net.parameters() if p.
    requires_grad)
248 print("\n\nThe number of learnable parameters in the model: %d" %
    number_of_learnable_params)
249 number_of_learnable_params = sum(p.numel() for p in bme_net_custom.parameters() if p.
    requires_grad)
250 print("\n\nThe number of learnable parameters in the model: %d" %
    number_of_learnable_params)
251
252 # training and testing
253 bme_net_custom.run_code_for_training(bme_net, net_name='base')
254 bme_net_custom.run_code_for_training(bme_net_custom, net_name='custom')
255
256 bme_net_custom.run_code_for_testing(bme_net, net_name='base')
257 bme_net_custom.run_code_for_testing(bme_net_custom, net_name='custom')
258
259 compare_loss()

```

5 Code: Task 2

```

1 from torch.utils.data import DataLoader, Dataset
2 import torch.nn as nn
3 from DLStudio import *
4 import seaborn as sns
5 import matplotlib.pyplot as plt
6 from PIL import Image
7 from torchvision import datasets, transforms
8
9 class CustomDataloader(Dataset):
10     def __init__(self, root, train=True):
11         self.img_paths = []
12         self.labels = []
13         if train:
14             self.root = os.path.join(root, 'train')
15             self.img_paths = [os.path.join(root, 'train', img) for img in os.listdir(os.
    path.join(root, 'train'))]
16             self.labels = np.load(f'{root}/train_labels.npy', allow_pickle=True)
17         else:
18             self.root = os.path.join(root, 'validation')

```

```

19         self.img_paths = [os.path.join(root, 'validation', img) for img in os.
20                             listdir(os.path.join(root, 'validation'))]
21         self.labels = np.load(f'{root}/val_labels.npy', allow_pickle=True)
22
23     def __len__(self):
24         return len(self.img_paths)
25
26     def __getitem__(self, index):
27         img_path = os.path.join(self.root, str(index)+'.jpg')
28         self.img = Image.open(img_path)
29         transform = transforms.Compose([
30             transforms.Grayscale(num_output_channels=3), # some images are Grayscale but
31                 we want images to be 3 X 64 X 64
32             transforms.ToTensor(),
33             transforms.Normalize((0.5,0.5,0.5),(0.5,0.5,0.5))
34         ])
35         transformed_img = transform(self.img)
36         label = self.labels[index]
37         return (transformed_img, label)
38
39 class HW5Net(nn.Module):
40     def __init__(self, dl_studio, skip_connections=True, depth=8):
41         super(HW5Net, self).__init__()
42
43     ##### Start: Copy from DLStudio with minor changes #####
44
45         self.dl_studio = dl_studio
46         self.depth = depth
47         image_size = dl_studio.image_size
48         num_ds = 0                                ## num_ds stands for number of
49             downsampling steps
50         self.conv = nn.Conv2d(3, 64, 3, padding=1)
51         self.skip64_arr = nn.ModuleList()
52         for i in range(self.depth):
53             self.skip64_arr.append(HW5Net.SkipBlock(64, 64, skip_connections=
54                 skip_connections))
55         self.skip64to128ds = HW5Net.SkipBlock(64, 128, downsample=True, skip_connections
56             =skip_connections )
57         num_ds += 1
58         self.skip128_arr = nn.ModuleList()
59         for i in range(self.depth):
60             self.skip128_arr.append(HW5Net.SkipBlock(128, 128, skip_connections=
61                 skip_connections))
62         self.skip128to256ds = HW5Net.SkipBlock(128, 256, downsample=True,
63             skip_connections=skip_connections )
64         num_ds += 1
65         self.skip256_arr = nn.ModuleList()
66         for i in range(self.depth):
67             self.skip256_arr.append(HW5Net.SkipBlock(256, 256, skip_connections=
68                 skip_connections))
69         self.fc1 = nn.Linear( (image_size[0]// (2 ** num_ds)) * (image_size[1]//(2 **
70             num_ds)) * 256, 1000)
71         self.fc2 = nn.Linear(1000, 10)
72
73     def forward(self, x):
74         x = nn.functional.relu(self.conv(x))
75         for skip64 in self.skip64_arr:
76             x = skip64(x)
77         x = self.skip64to128ds(x)
78         for skip128 in self.skip128_arr:
79             x = skip128(x)
80         x = self.skip128to256ds(x)
81         for skip256 in self.skip256_arr:
82             x = skip256(x)
83         x = x.view( x.shape[0], - 1 )
84         x = nn.functional.relu(self.fc1(x))
85         x = self.fc2(x)
86         return x
87
88 class SkipBlock(nn.Module):
89     def __init__(self, in_ch, out_ch, downsample=False, skip_connections=True):

```

```

83         super(HW5Net.SkipBlock, self).__init__()
84         self.downsample = downsample
85         self.skip_connections = skip_connections
86         self.in_ch = in_ch
87         self.out_ch = out_ch
88         self.conv1 = nn.Conv2d(in_ch, in_ch, 3, stride=1, padding=1)
89         self.conv2 = nn.Conv2d(in_ch, out_ch, 3, stride=1, padding=1)
90         self.bn1 = nn.BatchNorm2d(in_ch)
91         self.bn2 = nn.BatchNorm2d(out_ch)
92         self.in2out = nn.Conv2d(in_ch, out_ch, 1)
93         if downsample:
94             ## Setting stride to 2 and kernel_size to 1 amounts to retaining every
95             ## other pixel in the image --- which halves the size of the image:
96             self.downsampler1 = nn.Conv2d(in_ch, in_ch, 1, stride=2)
97             self.downsampler2 = nn.Conv2d(out_ch, out_ch, 1, stride=2)
98
99     def forward(self, x):
100         identity = x
101         out = self.conv1(x)
102         out = self.bn1(out)
103         out = nn.functional.relu(out)
104         out = self.conv2(out)
105         out = self.bn2(out)
106         out = nn.functional.relu(out)
107         if self.downsample:
108             identity = self.downsampler1(identity)
109             out = self.downsampler2(out)
110         if self.skip_connections:
111             if (self.in_ch == self.out_ch) and (self.downsample is False):
112                 out = out + identity
113             elif (self.in_ch != self.out_ch) and (self.downsample is False):
114                 identity = self.in2out(identity)
115                 out = out + identity
116             elif (self.in_ch != self.out_ch) and (self.downsample is True):
117                 out = out + torch.cat((identity, identity), dim=1)
118         return out
119 ##### End: Copy from DLStudio with minor changes #####
120 #####
121
122     def load_coco_dataset(self):
123         training_dataset = CustomDataLoader(root='output_dataset')
124         testing_dataset = CustomDataLoader(root='output_dataset', train=False)
125         self.train_data_loader = DataLoader(training_dataset, batch_size=self.dl_studio.
126             batch_size, shuffle=True)
127         self.test_data_loader = DataLoader(testing_dataset, batch_size=self.dl_studio.
128             batch_size, shuffle=False)
129 #####
130 ##### Start: Copy from DLStudio with minor changes #####
131     def run_code_for_training(self, net, net_name):
132         filename_for_out = "performance_numbers_" + net_name + ".txt"
133         FILE = open(filename_for_out, 'w')
134         net = copy.deepcopy(net)
135         net = net.to(self.dl_studio.device)
136         criterion = nn.CrossEntropyLoss()
137         # optimizer = optim.SGD(net.parameters(), lr=self.dl_studio.learning_rate,
138             momentum=self.dl_studio.momentum)
139         optimizer = optim.Adam(net.parameters(), lr=self.dl_studio.learning_rate, betas
140             =(0.9,0.99))
141         print("\n\nStarting training loop...")
142         start_time = time.perf_counter()
143         loss_tally = []
144         elapsed_time = 0.0
145         for epoch in range(self.dl_studio.epochs):
146             print("")
147             running_loss = 0.0
148             for i, data in enumerate(self.train_data_loader):
149                 inputs, labels = data
150                 if i % 50 == 49:
151                     current_time = time.perf_counter()
152                     elapsed_time = current_time - start_time
153                     print("\n\n[epoch:%d/%d iter=%4d elapsed_time=%5dsecs] Ground
154                         Truth:uuuuu" %

```

```

151         (epoch+1, self.dl_studio.epochs, i+1, elapsed_time) +
152         '\u'.join('%10s' % self.dl_studio.class_labels[labels[j]] for j
153                 in range(self.dl_studio.batch_size)))
154 inputs = inputs.to(self.dl_studio.device)
155 labels = labels.to(self.dl_studio.device)
156 optimizer.zero_grad()
157 outputs = net(inputs)
158 loss = criterion(outputs, labels)
159 running_loss += loss.item()
160 if i % 50 == 49:
161     _, predicted = torch.max(outputs.data, 1)
162     print("[epoch:%d/%d\uiter=%4d\uelapsed_time=%5d\usecs]\u\u\uPredicted\u
163           Labels:\u" %
164           (epoch+1, self.dl_studio.epochs, i+1, elapsed_time) +
165           '\u'.join('%10s' % self.dl_studio.class_labels[predicted[j]] for j
166                   in range(self.dl_studio.batch_size)))
167     avg_loss = running_loss / float(1000)
168     loss_tally.append(avg_loss)
169     print("[epoch:%d/%d\uiter=%4d\uelapsed_time=%5d\usecs]\u\u\uLoss:\u%.3f"
170           %
171           (epoch+1, self.
172             dl_studio.epochs,
173             i+1,
174             elapsed_time,
175             avg_loss))
176
177     FILE.write("%.3f\n" % avg_loss)
178     FILE.flush()
179     running_loss = 0.0
180     loss.backward()
181     optimizer.step()
182     print("\nFinished\uTraining\n")
183     self.save_model(net, net_name)
184     plt.figure(figsize=(10,5))
185     plt.title(net_name + ":\uLabeling\uLoss\uvs.\uIterations")
186     plt.plot(loss_tally)
187     plt.xlabel("iterations")
188     plt.ylabel("loss")
189     plt.legend(["Plot\uof\uloss\uversus\uiterations"], fontsize="x-large")
190     plt.savefig(f"loss_vs_iterations_{net_name}.png")
191
192 def save_model(self, model, net_name):
193     torch.save(model.state_dict(), self.dl_studio.path_saved_model + '_' + net_name)
194
195 def run_code_for_testing(self, net, net_name):
196     net.load_state_dict(torch.load(self.dl_studio.path_saved_model + '_' + net_name)
197 )
198     net = net.eval()
199     net = net.to(self.dl_studio.device)
200     filename_for_results = "classification_results_" + net_name + str(self.dl_studio
201     .epochs) + ".txt"
202     FILE = open(filename_for_results, 'w')
203     correct = 0
204     total = 0
205     confusion_matrix = torch.zeros(len(self.dl_studio.class_labels), len(self.
206     dl_studio.class_labels))
207     class_correct = [0] * len(self.dl_studio.class_labels)
208     class_total = [0] * len(self.dl_studio.class_labels)
209     with torch.no_grad():
210         for i, data in enumerate(self.test_data_loader):
211             ## data is set to the images and the labels for one batch at a time:
212             images, labels = data
213             images = images.to(self.dl_studio.device)
214             labels = labels.to(self.dl_studio.device)
215             if i % 50 == 49:
216                 print("\n\n[i=%d:]\uGround\uTruth:\u\u\u\u" % (i+1) + '\u'.join('%5s' %
217                 self.dl_studio.class_labels[labels[j]] for j in range(self.
218                 dl_studio.batch_size)))
219             outputs = net(images)
220             _, predicted = torch.max(outputs.data, 1)#
221             if i % 50 == 49:
222                 print("[i=%d:]\uPredicted\uLabels:\u" % (i+1) + '\u'.join('%5s' % self.
223                 dl_studio.class_labels[predicted[j]] for j in range(self.
224                 dl_studio.batch_size)))

```

```

209         logger = logging.getLogger()
210         old_level = logger.level
211
212         for label, prediction in zip(labels, predicted):
213             confusion_matrix[label][prediction] += 1
214         total += labels.size(0)
215         correct += (predicted == labels).sum().item()
216         ## comp is a list of size batch_size of "True" and "False" vals
217         comp = predicted == labels
218         for j in range(len(labels)):
219             label = labels[j]
220             class_correct[label] += comp[j].item()
221             class_total[label] += 1
222     for j in range(len(self.dl_studio.class_labels)):
223         print('Prediction accuracy for %5s: %2d%%' % (self.dl_studio.class_labels[
224             j], 100 * class_correct[j] / class_total[j]))
225         FILE.write('\n\nPrediction accuracy for %5s: %2d%%\n' % (self.dl_studio.
226             class_labels[j], 100 * class_correct[j] / class_total[j]))
227     print("\n\nOverall accuracy of the network on the 10000 test images: %d%%" %
228         (100 * correct / float(total)))
229     FILE.write("\n\nOverall accuracy of the network on the 10000 test images: %d%%\n" %
230         (100 * correct / float(total)))
231     print("\n\nDisplaying the confusion matrix:\n")
232     FILE.write("\n\nDisplaying the confusion matrix:\n\n")
233     out_str = "UUUUUUUUUU"
234     for j in range(len(self.dl_studio.class_labels)): out_str += "%7s" % self.
235         dl_studio.class_labels[j]
236     print(out_str + "\n")
237     FILE.write(out_str + "\n\n")
238     for i, label in enumerate(self.dl_studio.class_labels):
239         out_percents = [100 * confusion_matrix[i, j] / float(class_total[i])
240             for j in range(len(self.dl_studio.
241                 class_labels))]
242         out_percents = ["%.2f" % item.item() for item in out_percents]
243         out_str = "%6s:UU" % self.dl_studio.class_labels[i]
244         for j in range(len(self.dl_studio.class_labels)): out_str += "%7s" %
245             out_percents[j]
246         print(out_str)
247         FILE.write(out_str + "\n")
248     FILE.close()
249     self.class_correct = class_correct
250     self.class_total = class_total
251     self.correct = correct
252     self.total = total
253     self.display_conf_matrix(confusion_matrix, net_name)
254     ##### End: Copy from DLStudio with minor changes #####
255     #####
256
257     def display_conf_matrix(self, confusion_matrix, net_name):
258         accuracy_matrix = []
259         for i, label in enumerate(self.dl_studio.class_labels):
260             out_percents = [100 * confusion_matrix[i, j] / float(self.class_total[i])
261                 for j in range(len(self.dl_studio.
262                     class_labels))]
263             out_percents = [item.item() for item in out_percents]
264             accuracy_matrix.append([out_percents[j] for j in range(len(self.dl_studio.
265                 class_labels))])
266         plt.figure(figsize=(6, 5))
267         sns.heatmap(np.array(accuracy_matrix, dtype=float), annot=True, fmt='%.2f', cmap=
268             'Blues', cbar=True,
269             xticklabels=self.dl_studio.class_labels, yticklabels=self.dl_studio.
270                 class_labels)
271         plt.savefig(f'{net_name}_Confusion_matrix.png', bbox_inches='tight')
272
273     dls = DLStudio(
274         dataroot = "./output_dataset",
275         image_size = [64, 64],
276         path_saved_model = "./saved_model",
277         momentum = 0.9,
278         learning_rate = 1e-5,
279         epochs = 12,
280         batch_size = 64,

```

```

271         classes = ('airplane', 'bus', 'cat', 'dog', 'pizza'),
272         use_gpu = True,
273     )
274
275 net = HW5Net(dls, skip_connections=True, depth=8)
276 net.load_coco_dataset()
277
278 net.run_code_for_training(net, net_name='HW5Net')
279 net.run_code_for_testing(net, net_name='HW5Net')

```

6 Code: Generating dataset

```

1  from pycocotools.coco import COCO
2  import os
3  from PIL import Image
4  import sys
5  import matplotlib.pyplot as plt
6  from torchvision import transforms
7  from tabulate import tabulate
8  import numpy as np
9
10 # Set COCO dataset paths
11 img_dir = 'train2014'
12 ann_file = 'annotations/instances_train2014.json'
13 output_dir = 'output_dataset'
14
15 # Load COCO dataset
16 coco = COCO(ann_file)
17
18 # Ensure output directories exist
19 os.makedirs(output_dir, exist_ok=True)
20
21 def extract_imgs(categories):
22     cat_ids = coco.getCatIds()
23     cat_names = [cat for cat in categories]
24     train_labels = []
25     val_labels = []
26     count_train = 0
27     count_val = 0
28     obj_count_val = [4, 6, 5, 5, 9]
29     for label, cat_name in enumerate(cat_names):
30         cat_id = coco.getCatIds(catNms=cat_name)
31         img_ids = coco.getImgIds(catIds=cat_id)
32
33         suitable_img_ids = []
34
35         for i, img_id in enumerate(img_ids):
36             img_info = coco.loadImgs(img_id)[0]
37             ann_ids = coco.getAnnIds(imgIds=img_id, iscrowd=False)
38             anns = coco.loadAnns(ann_ids)
39             obj_counts = {}
40
41             for ann in anns:
42                 obj_category = coco.loadCats(ann['category_id'])[0]['name']
43                 obj_counts[obj_category] = obj_counts.get(obj_category, 0) + 1
44
45             other_cats = [cat for cat in categories if cat != cat_name]
46             if len(obj_counts) == 1:
47                 suitable_img_ids.append(img_id)
48             elif all(item not in obj_counts for item in other_cats) and len(obj_counts)
49                 < obj_count_val[label]:
50                 suitable_img_ids.append(img_id)
51
52 if len(suitable_img_ids) < 2000:
53     print(f"Warning: not enough image for class {cat_name}. Found {len(
54         suitable_img_ids)}, required 2000.")
55 else:
56     print(f'There is sufficient images for class {cat_name}. Found {len(
57         suitable_img_ids)}, required 2000.')

```

```

56     suitable_img_ids = suitable_img_ids[:2000]
57
58     for i, suitable_img_id in enumerate(suitable_img_ids):
59         img_info = coco.loadImgs(suitable_img_id)[0]
60         if i < 1500:
61             train_labels.append(label)
62             save_image(img_info, count_train)
63             count_train += 1
64         else:
65             val_labels.append(label)
66             save_image(img_info, count_val, train=False)
67             count_val += 1
68
69     np.save(f'{output_dir}/train_labels.npy', np.array(train_labels, dtype=object),
70           allow_pickle=True)
71     np.save(f'{output_dir}/val_labels.npy', np.array(val_labels, dtype=object),
72           allow_pickle=True)
73
74 def save_image(img_info, count, train=True):
75     img_path = os.path.join(img_dir, img_info['file_name'])
76     if train:
77         save_dir = os.path.join(output_dir, 'train')
78     else:
79         save_dir = os.path.join(output_dir, 'validation')
80     os.makedirs(save_dir, exist_ok=True)
81
82     img = Image.open(img_path).resize((64, 64))
83     img.save(os.path.join(save_dir, str(count) + '.jpg'))
84
85 def dataset_visualization(categories, type):
86     fig, axes = plt.subplots(5,3)
87
88     for row, cat in enumerate(categories):
89         for col in range(3):
90             if type == 'train':
91                 img = Image.open(os.path.join('output_dataset', 'train', str(row*1500+col
92                 +1)+'.jpg'))
93             else:
94                 img = Image.open(os.path.join('output_dataset', 'validation', str(row*500+
95                 col+1)+'.jpg'))
96             axes[row][col].imshow(img)
97             axes[row][col].axis('off')
98             fig.text(0.08, 0.9 - (row + 0.5) / 6, cat, ha='center', va='center', fontsize
99             =12, rotation=0)
100     plt.suptitle(f'{type}')
101     plt.savefig(f'{type}.jpg')
102     plt.close(fig)
103
104 # cat_ids = extract_imgs(['airplane', 'bus', 'cat', 'dog', 'pizza'])
105 dataset_visualization(['airplane', 'bus', 'cat', 'dog', 'pizza'], 'train')
106 dataset_visualization(['airplane', 'bus', 'cat', 'dog', 'pizza'], 'validation')

```
